



Universitat d'Alacant
Universidad de Alicante

Práctica 01

Sesión 1 - Introducción, entorno de trabajo y clase Variable



Nombre: Daironas Sukis

NIE: Y1759578T

Grupo: 01

Asignatura: Sistemas Inteligentes, Ingeniería Informática, Universidad de Alicante

1. Contexto

Se ha escogido la versión con la plantilla que usa tkinter, y para esta primera sesión se ha añadido el archivo **variable.py**, donde se guardará la implementación de la clase variable. El archivo **main.py** ha sido modificado como podremos ver a continuación.

2. Creación de la Clase Variable

Los atributos añadidos a la clase Variable han sido:

```
class Variable:
    def __init__(self, nombre, tipo, fila, col, longitud, dominio):
        self.nombre = nombre
        self.tipo = tipo          # h/v
        self.fila = fila
        self.col = col
        self.longitud = longitud
        self.dominio = dominio
        self.restricciones = []
```

- **nombre**: para identificar la Variable
- **tipo**: puede ser horizontal o vertical
- **fila, col**: para indicar en qué posición empieza la Variable
- **longitud**
- **dominio**: las palabras que pueden entrar en esa Variable, serán las que coincidan exactamente con su longitud, y en caso de que hubiese un carácter en esa variable, que coincida la posición del carácter con el de la palabra
- **restricciones**: esta no se necesita en esta primera sesión, pero el enunciado nos pide este atributo

El método **__str__** implementado, devuelve un string en formato específico de la clase Variable.

```
def __str__(self):
    return f"Nombre {self.nombre} Posición {self.fila} {self.col} Tipo: {self.tipo} Dominio: {self.dominio}"
```

3. Modificación del main.py

Una vez implementada la clase Variable, necesitamos una función que recorra la tabla en busca de las Variables disponibles. Para ello se ha implementado dos funciones principales, una que recorre por filas la tabla, en búsqueda de Variables horizontales, y otra que recorre por columnas las tablas, en búsqueda de variables verticales.

3.1 Función ConstruyeVariablesHorizontales

3.1.1 Código

```
def construyeVariablesHorizontal(tablero, almacen):
    variables = []
    contador = 0

    for fila in range(tablero.getAlto()):
        longitudVariable = 0
        col_inicio = 0
        for col in range(tablero.getAncho()):
            if tablero.getCelda(fila,col) != LLENA:
                if longitudVariable == 0:
                    col_inicio = col
                    longitudVariable += 1

                elif longitudVariable >= 2:
                    # Evitamos crear variables de un solo
hueco
                    # Creamos la variable
                    dominio = filtraDominioHorizontal(tablero, almacen, fila, col_inicio,
longitudVariable)
                    nuevaVariable = Variable(contador, "H", fila, col_inicio, longitudVariable,
dominio)
                    variables.append(nuevaVariable)
                    contador += 1
                    longitudVariable = 0
                else:
                    longitudVariable = 0

            if longitudVariable >= 2:
                dominio = filtraDominioHorizontal(tablero, almacen, fila, col_inicio,
longitudVariable)
                nuevaVariable = Variable(contador, "H", fila, col_inicio, longitudVariable, dominio)
                variables.append(nuevaVariable)
                contador += 1
                longitudVariable = 0

    return variables, contador
```

3.1.2 Explicación

Esta función recibe como parámetros de entrada el tablero a recorrer, y el almacén, que es un array de palabras que se van a usar en el crucigrama. También inicializamos un Array de variables vacío que servirá para almacenar todas las variables horizontales que vamos a encontrar.

Una vez inicializado los atributos necesarios, se procede a recorrer el tablero mediante dos bucles for anidados, e inicializamos las Variables "*longitudVariable*", para calcular cuántos caracteres va a tener, y "*col_inicio*", para guardar en qué columna empieza dicha Variable.

Recorriendo celda por celda, necesitamos comprobar los siguientes casos para identificar una Variable a guardar:

- **Caso 1: Si la celda no está llena**

Es decir si está vacía o contiene una letra. En este caso sabemos que posiblemente pertenezca a una Variable (a menos que solo ocupe una celda), y si justo en esa iteración *longitudVariable* es igual a cero, pues sabemos que es el comienzo de una variable, por lo que asignamos *col_inicio* al número de columna actual e incrementamos en 1 el valor de *longitudVariable* actual.

- **Caso 2: Si longitudVariable es mayor o igual que 2**

En esta condición ya sabemos que la celda está llena, y comprobamos que la longitud de la variable es mayor o igual que dos, para evitar que se creen variables de una sola celda.

Con la ayuda de la función auxiliar *filtraDominioHorizontal*, obtenemos el dominio de esa variable. Posteriormente se procede a crear dicha variable y se añade a la lista de variables, incrementando también el contador para seguir el orden del id, y se restablece la longitud para la siguiente variable.

- **Caso 3: longitudVariable vale 1 o menos**

Esta es la condición en la que sabemos que la celda está llena y además *longitudVariable* es menor o igual a 1, de esta manera no guardamos la variable y reiniciamos el contador *longitudVariable*.

Al finalizar el bucle for que recorre la fila entera, nos aseguramos de que no se haya quedado a medias una Variable porque justo la última iteración si que era una celda que podía pertenecer. Por lo que comprobamos si *longitudVariable* es mayor o igual que 2, y creamos de nuevo la variable.

Una vez comprobadas todas las celdas, devolvemos la lista de variables y el contador

3.1.3 Función auxiliar filtraDominioHorizontal

```
def filtraDominioHorizontal(tablero, almacen, fila, col_inicio, longitud):
    dominio = almacen.get(longitud, set()).copy()

    for i in range(longitud):
        celda = tablero.getCelda(fila, col_inicio + i)
        if celda != VACIA and celda != LLENA:
            nuevoFiltrado = []
            for palabra in dominio:
                if palabra[i] == celda:
                    nuevoFiltrado.append(palabra)
            dominio = nuevoFiltrado

    return dominio
```

Esta función se encarga de obtener con el método **.get()**, todas aquellas palabras guardadas en el almacén, que tenga una cantidad de caracteres establecida por la variable *“longitud”*. El bucle for anidado, se encarga de filtrar la lista de palabras, de manera que si encuentra una celda con un carácter, se asegura de que el dominio sólo esté compuesto por palabras que contengan dicho carácter en su posición específica.

3.2 Función ConstruyeVariablesVertical

3.2.1 Explicación

Esta función funciona de la misma manera que la explicada anteriormente, lo único que lo diferencia es que el recorrido lo hace de manera vertical, y se le pasa en contador como parámetro, para seguir el orden de los nombres de las Variables.

3.2.2 Código

```
def construyeVariablesVertical(tablero, almacen, contador): # Aqui pasamos el
contador como parametro para evitar que el nombre de las variables se solapen con los de la
funcion "construyeVariablesHorizontal"
    variables = []

    for col in range(tablero.getAncho()):
        longitudVariable = 0
        fila_inicio = 0
        for fila in range(tablero.getAlto()):
            if tablero.getCelda(fila,col) != LLENA:
                if longitudVariable == 0:
                    fila_inicio = fila
                    longitudVariable += 1

            elif longitudVariable >= 2: # Evitamos crear variables de un solo
hueco
```

```

        # Creamos la variable
        dominio = filtraDominioVertical(tablero, almacen, fila_inicio, col,
longitudVariable)

        nuevaVariable = Variable(contador, "V", fila_inicio, col, longitudVariable,
dominio)

        variables.append(nuevaVariable)
        contador += 1
        longitudVariable = 0
    else:
        longitudVariable = 0

    if longitudVariable >= 2:
        dominio = filtraDominioVertical(tablero, almacen, fila_inicio, col,
longitudVariable)
        nuevaVariable = Variable(contador, "V", fila_inicio, col, longitudVariable, dominio)
        variables.append(nuevaVariable)
        contador += 1
        longitudVariable = 0

    return variables

```

3.2.3 Función auxiliar filtraDominioVertical

Funciona también de la misma manera, solo que el recorrido es vertical

```

def filtraDominioVertical(tablero, almacen, fila_inicio, col, longitud):
    dominio = almacen.get(longitud, set()).copy()

    for i in range(longitud):
        celda = tablero.getCelda(fila_inicio + i, col)
        if celda != VACIA and celda != LLENA:
            nuevoFiltrado = []
            for palabra in dominio:
                if palabra[i] == celda:
                    nuevoFiltrado.append(palabra)
            dominio = nuevoFiltrado

    return dominio

```

3.3 Modificación de la función main

Se ha añadido código para poder ver visualmente el botón VAR, y hacerlo funcional a la hora de pulsarlo.

```
posBotBK=altoVentana//5-altoBoton//2
posBotFC=(altoVentana//5)*2-altoBoton//2
posBotAC3=(altoVentana//5)*3-altoBoton//2
posBotVariable=(altoVentana//5)*4-altoBoton//2

botBK=pygame.Rect(anchoVentana-95, posBotBK, 70, altoBoton)
botFC=pygame.Rect(anchoVentana-95,posBotFC , 70, altoBoton)
botAC3=pygame.Rect(anchoVentana-95, posBotAC3, 70, altoBoton)
botVariable=pygame.Rect(anchoVentana-95, posBotVariable, 70, altoBoton)
```

```
pintarBoton(screen, fuenteBot, botBK, "BK")
pintarBoton(screen, fuenteBot, botFC, "FC")
pintarBoton(screen, fuenteBot, botAC3, "AC3")
pintarBoton(screen, fuenteBot, botVariable, "VAR")
```

También se ha modificado la condición del if, para detectar cuando se pulsa el botón de Variable, y así poder obtener la lista de variables para imprimirlas en consola.

```
elif pulsaBoton(pos, botVariable):
    variablesHorizontales, contador = construyeVariablesHorizontal(tablero,
almacen)

    variablesVerticales = construyeVariablesVertical(tablero, almacen, contador)
    todasVariables = variablesHorizontales + variablesVerticales
    for v in todasVariables:
        print(v)
```

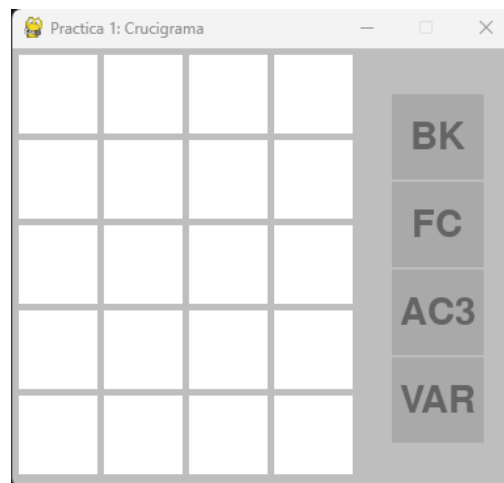
4. Ejecución del crucigrama

Para ejecutar el crucigrama y poder comprobar el trabajo realizado debemos de abrir una consola, y situados al nivel de los archivos del código fuente ejecutar:

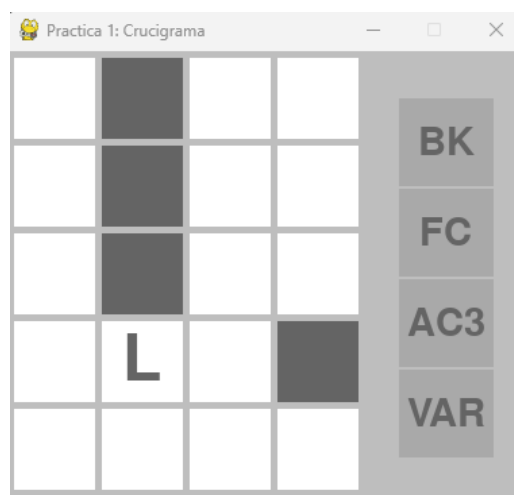
```
python main.py --filas M --columnas N --fich fichero.txt
```

```
PS C:\Users\Usuario\Desktop\SistemasInteligentes\Practica01> py main.py --filas 5 --columnas 4 --fich d0.txt
pygame 2.6.1 (SDL 2.28.4, Python 3.13.15)
Hello from the pygame community. https://www.pygame.org/contribute.html
Uso: python main.py --filas M --columnas N --dic archivo.txt
```

Una vez ejecutado se abrirá la interfaz gráfica del crucigrama, donde podremos manipularlo a nuestro antojo pulsando el click izquierdo para ocupar una celda, y el click derecho para poder insertar un carácter.



Modificando el crucigrama para probar un ejemplo quedaría así:



Y al pulsar el botón VAR, en la consola se imprimiría la siguiente lista de Variables:

```
Nombre 0 Posición 0 2 Tipo: H Dominio: {'DE', 'NO', 'AR', 'ES', 'SO', 'AS', 'SI', 'AL', 'LA'}
Nombre 1 Posición 1 2 Tipo: H Dominio: {'DE', 'NO', 'AR', 'ES', 'SO', 'AS', 'SI', 'AL', 'LA'}
Nombre 2 Posición 2 2 Tipo: H Dominio: {'DE', 'NO', 'AR', 'ES', 'SO', 'AS', 'SI', 'AL', 'LA'}
Nombre 3 Posición 3 0 Tipo: H Dominio: ['OLA']
Nombre 4 Posición 4 0 Tipo: H Dominio: {'LISO', 'MORA', 'PERO', 'ROSA', 'OSOS', 'OLOR'}
Nombre 5 Posición 0 0 Tipo: V Dominio: {'CIRCO', 'ASILO', 'RETOS', 'FOCOS', 'SETOS', 'ESOP', 'OSERA', 'TOTEM'}
Nombre 6 Posición 3 1 Tipo: V Dominio: ['LA']
Nombre 7 Posición 0 2 Tipo: V Dominio: {'CIRCO', 'ASILO', 'RETOS', 'FOCOS', 'SETOS', 'ESOP', 'OSERA', 'TOTEM'}
Nombre 8 Posición 0 3 Tipo: V Dominio: {'CON', 'SOL', 'OLA', 'OSO', 'ARA', 'RON'}
```